

你很快就可以使用了。立即注册，开始你的 Notion

Week 4: Attention

Reading

- [Online normalizer calculation for softmax.](#)
- [FlashAttention: Fast and Memory-Efficient Exact Attention in Only One Forward Pass.](#)
- [FlashAttention-2: Faster Attention with Better Parallelism, Model Support, and Implementation.](#)
 - Enhancements over V1:
 - **This week: Restructuring of the core Algorithm 1.**
 - Next week: Use tensor cores
 - Next week: Use warp-level work partitioning
- [Optional] [How to read a paper.](#)
- [Optional] [How to Read an Engineering Research Paper](#)

Note: FlashAttentionV1 paper is good to read for background, but the implementation of Algorithm 1 in the paper is awkward to implement. The Algorithm 1 in FlashAttention-2 is better.

Thus, we will focus our initial implementation efforts on Algorithm 1 in FlashAttention-2 (and then move on to the more advanced aspects of FlashAttention-2, such as tensor cores, work partitioning, etc).

Assignment

1. Using *unparallelized C*, implement Algorithm 1 in Section 2.1 of the FlashAttention-2 paper.
2. Using *parallelized CUDA*, implement Algorithm 1 in Section 2.1 of the FlashAttention-2 paper.

The goal of this assignment is to implement a *correct, parallelized implementation* of Algorithm 1. We will not address lower level optimization for now (e.g. tensor cores, work partitioning, etc) and address those later.

Discussion

Definition of attention:

$$O = \text{Attention}(Q, K, V) = \text{softmax}$$

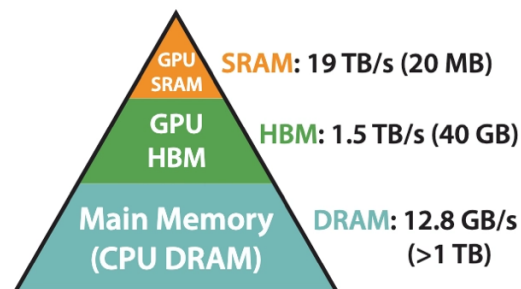
$$O, Q, K, V \in \mathbb{R}^{N \times d}$$

Mathematical operations:

- Matrix multiplication
- Transpose
- Scalar multiplication of a matrix
- Softmax

Definition of Softmax:

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}}$$



**Memory Hierarchy with
Bandwidth & Memory Size**

Shared memory, size

FlashAttention takes the shared memory tiling idea for matrix multiplication by leveraging the online softmax trick.

Key correctness property (similar to matrix multiplication): each threadblock only updates a part of the output matrix O in the HBM.

Approach:

- Divide N into B_r -sized chunks.
- Each input chunk of Q_i of Q is assigned to a threadblock
 - This threadblock is responsible for the output chunk O_i
 - This threadblock iterates over every chunk of K
 - Thus, every threadblock is going to read every chunk of K

chunk = block = tile

How do we calculate B_r and B_c which are inputs into flashattention?

- These two parameters are constrained by the amount of memory a threadblock can hold (less than 1MB).

Line 3 launches a kernel with the T_r threadblocks.

Line 4 requires a single Q_i in shared memory.

- Total of $B_r \times d \times 4$ bytes

Line 5 requires a single O_i in shared memory.

- Total of $B_r \times d \times 4$ bytes

Line 7 requires a single K_j in shared memory and a single V .

- $B_c \times d \times 4$ bytes + $B_c \times d \times 4$ bytes
- Total of $2 \times B_c \times d \times 4$ bytes

Thus, we have a constraint that:

- $B_r \times d \times 4 \text{ bytes} + B_r \times d \times 4 + 2 \times B_c \times d \times 4$ memory in a single threadblock - the size of the merge

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(\mathbf{S}_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)}))$ (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \mathbf{V}_j$
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

$\text{diag}(\dots)$

$\text{diag}(X)$ turns a vector X with shape $(m, 1)$ into a diagonal matrix where each matrix element on the diagonal at (i, i) is set to the value of X_i .

Thus, $\text{diag}(X)M$ scales each row of the matrix M by the corresponding element in X .

This is a common trick in linear algebra to apply element-wise scaling to a matrix. It is often written in compact notation.

$\text{diag}(\dots)^{-1}$

To understand what $Z = \text{diag}(X)^{-1}Y$ does, see its implementation in

`diag_inv_scale`.

```
// X has shape (m, 1) // Y has shape (m, n) //
diag_inv_scale(float* X, float* Y, float* Z, in
m; ++i) { float inv_xi = 1.0f / x[i]; for (int
= Y[i * n + j] * inv_xi; } } }
```

Example:

```
X = [2, 4, 5] Y = [ [10, 20], [12, 8], [5, 15]
```

Then output will be:

```
[ [10/2, 20/2] = [5.0, 10.0], [12/4, 8/4] = [3.
]
```

Online Softmax

Lines 10-12 in the flash attention algorithm above implement
Online normalizer calculation for softmax, Sections 1-4

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}}$$

Algorithm 3 Safe softmax with online normalizer calculation

```
1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
9: end for
```

Calculating B_c, B_r from M

```
typedef struct { tile Kj; // (B_c, d) tile Vj;
tile Oi; // (B_r, d) tile li; // (B_r, 1) tile
B_c) tile mij; // (B_r, 1) tile Pij; // (B_r, B
mi_new; // (B_r, 1) tile li_new; // (B_r, 1) }
```

Using the above struct, we derive an inequality between

$$M < 2 * B_c * d + 2 * B_r * d + 6 *$$

FlashAttentionV2: Algorithm 1